

SOF 103 Week 10: Objects and Classes

Generated by Review Agent on 2026-07-02 06:36 UTC

Source: SOF 103-Chapter 10 - Objects and Classes2024_09.ppt

Classification confidence: 80%

1. Core Knowledge Outline

中文大纲

- **面向对象编程 (OOP) 基础**

- 核心思想：将现实世界的事物抽象为程序中的“对象”。
- 为什么重要：提高代码的**可重用性**、**可维护性**和**可扩展性**，使复杂程序更易于管理。

- **类 (Class)**

- **定义**：类是创建对象的“蓝图”或“模板”。它定义了对象将拥有什么（属性/数据）和能做什么（方法/行为）。
- **类比**：就像一张房屋设计图。设计图本身不是房子，但可以根据它建造出许多结构相同的房子。

- **对象 (Object)**

- **定义**：对象是类的一个具体“实例”。它拥有类中定义的属性和方法，并且这些属性有具体的值。
- **类比**：根据一张房屋设计图建造出来的那栋具体的房子。每栋房子（对象）都有颜色、地址等具体属性。

- **类与对象的关系**

- 类是抽象的，对象是具体的。
- 一个类可以创建多个不同的对象。
- 对象是程序运行时的基本单元。

- **类的组成**

- **属性 (Attributes/Fields):** 描述对象状态的数据。例如, 一个 `Car` 类可以有 `color`、`speed`、`model` 等属性。
- **方法 (Methods):** 定义对象可以执行的操作。例如, `Car` 类可以有 `start()`, `accelerate()`, `brake()` 等方法。
- **创建和使用对象**
 - **声明:** 声明一个对象变量, 类型为类名。
 - **实例化:** 使用 `new` 关键字创建对象实例。
 - **初始化:** 通过**构造方法 (Constructor)** 为对象的属性赋初值。
 - **使用:** 通过 `.` 运算符访问对象的属性和调用其方法。

English Outline

- **Object-Oriented Programming (OOP) Fundamentals**
 - Core Idea: Abstract real-world entities into programmatic "objects".
 - Why Important: Enhances code **reusability**, **maintainability**, and **scalability**, making complex programs easier to manage.
- **Class**
 - **Definition:** A class is a "blueprint" or "template" for creating objects. It defines what data (attributes) and behaviors (methods) an object will have.
 - **Analogy:** Like a blueprint for a house. The blueprint itself is not a house, but it can be used to build many identical houses.
- **Object**
 - **Definition:** An object is a concrete "instance" of a class. It has the attributes and methods defined by its class, with specific values for its attributes.
 - **Analogy:** The specific house built from that blueprint. Each house (object) has concrete properties like color and address.
- **Relationship between Class and Object**
 - The class is abstract; the object is concrete.
 - One class can create many different objects.
 - Objects are the fundamental runtime units of a program.

- **Components of a Class**

- **Attributes (Fields):** Data that describes the state of an object. E.g., a `Car` class can have attributes like `color` , `speed` , `model` .
- **Methods:** Actions that an object can perform. E.g., a `Car` class can have methods like `start()` , `accelerate()` , `brake()` .

- **Creating and Using Objects**

- **Declaration:** Declare an object variable with the class name as its type.
- **Instantiation:** Use the `new` keyword to create an instance of the class.
- **Initialization:** Assign initial values to the object's attributes via a **Constructor**.
- **Usage:** Access an object's attributes and call its methods using the `.` operator.

2. Key Concepts & Glossary

Term	Definition	Memory Anchor / Analogy
类 (Class)	定义对象结构和行为的蓝图或模板。它不占用内存空间，直到被实例化。 / A blueprint or template that defines the structure and behavior of objects. It does not occupy memory until instantiated.	房屋设计图 ：设计图本身不是房子，但定义了房子的结构和功能（房间数量、门窗位置）。 / House Blueprint : The blueprint itself isn't a house, but it defines the structure and features (number of rooms, window locations).
对象 (Object)	类的具体实例。它拥有类中定义的属性和方法，并在内存中占有空间。 / A concrete instance of a class. It possesses the attributes and methods defined by its class and occupies space in memory.	具体的房子 ：根据设计图建造出来的那栋房子，有具体的地址、颜色和住户。 / The Actual House : The specific building constructed from the blueprint, with a real address, color, and occupants.
属性 (Attribute/Field)	对象中用于存储数据的变量，描述了对象的状态。 / Variables within an object that store data, describing the object's state.	房子的特征 ：比如房子的颜色、面积、楼层数。 / House Features : Such as the house's color, area, and number of floors.
方法 (Method)	对象中定义的函数，描述了对象可以执行的操作或行为。 / Functions defined within an object that describe the actions or behaviors it can perform.	房子的功能 ：比如开门、开灯、开空调。 / House Functions : Such as opening the door, turning on the lights, or starting the AC.
实例化 (Instantiation)	使用 <code>new</code> 关键字从类创建对象的过程。 / The process of creating an object from a class using the <code>new</code> keyword.	按图施工 ：建筑队按照设计图开始建造房子的过程。 / Construction : The process of a construction team building a house according to the blueprint.
构造方法 (Constructor)	一种特殊的方法，在对象被实例化时自动调用，用于初始化对象的属性。 / A special method automatically called when an object is instantiated, used to initialize the object's attributes.	房子的初始配置 ：房子建好后，开发商进行的初始设置，比如接通水电、安装门锁。 / Initial Setup : The initial configuration done by the developer after the house is built, like connecting water and electricity, installing locks.

Term	Definition	Memory Anchor / Analogy
封装 (Encapsulation)	将数据（属性）和操作数据的方法捆绑在一起，并对外部隐藏内部实现细节的机制。 / The mechanism of bundling data (attributes) and methods that operate on the data, hiding internal implementation details from the outside.	房子的墙壁和门 ：墙壁隐藏了内部的电线和管道，只通过门和开关与外界交互。 / Walls and Doors : Walls hide the internal wiring and pipes, only allowing interaction through doors and switches.
new 关键字	用于在堆内存中为对象分配空间并调用构造方法的Java关键字。 / A Java keyword used to allocate memory for an object on the heap and invoke its constructor.	“开工”命令 ：建筑队听到“开工”命令后，才开始动工建造房子。 / "Start Construction" Command : The command that tells the construction team to begin building the house.

3. Critical Takeaways

中文总结

- 类是蓝图，对象是实物**：这是最核心的概念。永远记住，`class` 是设计图，`object` 是根据设计图造出来的具体东西。考试中区分二者是基础。
- new 是关键**：创建对象的唯一方式就是使用 `new` 关键字。`new` 会触发构造方法的执行，并在内存中为对象分配空间。
- 构造方法用于初始化**：构造方法的名字必须和类名完全相同，并且没有返回值。它的主要任务就是给对象的属性赋初始值。一个类可以有多个构造方法（重载）。
- 通过 . 访问成员**：创建对象后，使用 `对象名.属性名` 或 `对象名.方法名()` 来访问对象的属性和方法。
- 常见误区**：混淆类和对象；忘记使用 `new` 关键字；构造方法写错名字或返回值类型。

English Summary

- Class is Blueprint, Object is Reality**: This is the most fundamental concept. Always remember: `class` is the blueprint, `object` is the concrete thing built

from it. Distinguishing them is basic knowledge for exams.

2. **new is Key:** The only way to create an object is by using the `new` keyword. `new` triggers the execution of the constructor and allocates memory for the object.
 3. **Constructors are for Initialization:** A constructor's name must exactly match the class name and has no return type. Its main job is to assign initial values to an object's attributes. A class can have multiple constructors (overloading).
 4. **Access Members with `.`:** After creating an object, use `objectName.attributeName` OR `objectName.methodName()` to access its attributes and methods.
 5. **Common Mistakes:** Confusing a class with an object; forgetting to use the `new` keyword; writing a constructor with the wrong name or a return type.
-

4. Detailed Notes (AI-Expanded)

4.1 为什么需要面向对象？ / Why Do We Need OOP?

中文

想象一下，你要编写一个程序来管理一个图书馆的所有书籍。如果使用传统的面向过程编程，你可能会创建很多独立的变量，比如 `book1Title`, `book1Author`, `book1ISBN`, `book2Title`, `book2Author`, `book2ISBN` ... 这样代码会变得非常冗长、混乱，难以维护。如果你想增加一本书的“出版年份”信息，你需要在代码的各个地方添加新的变量。

面向对象编程（OOP）就是为了解决这个问题而生的。它允许我们将数据和操作数据的方法“封装”在一起，形成一个独立的单元——**对象**。对于图书馆的例子，我们可以创建一个 `Book` 类，它包含了 `title`, `author`, `isbn` 等**属性**（数据），以及 `checkOut()`, `returnBook()` 等**方法**（操作）。然后，每一本具体的书（如《三体》）就是这个 `Book` 类的一个**对象**。

这样做的好处是：

- * **模块化**：代码被组织成独立的、可管理的模块（对象）。
- * **可重用性**：`Book` 类可以被用来创建任意数量的书籍对象，无需重复编写代码。
- * **可维护性**：如果需要修改书籍的行为，只需要修改 `Book` 类本身，所有基于该类创建

的对象都会自动更新。

* **可扩展性**: 可以轻松地添加新的类 (如 `Magazine`, `DVD`) 来扩展程序的功能。

English

Imagine you need to write a program to manage all the books in a library. Using traditional procedural programming, you might create many independent variables like `book1Title`, `book1Author`, `book1ISBN`, `book2Title`, `book2Author`, `book2ISBN` ... This code becomes very long, messy, and hard to maintain. If you want to add a "publication year" for a book, you'd have to add new variables in many places throughout your code.

Object-Oriented Programming (OOP) was created to solve this problem. It allows us to "encapsulate" data and the methods that operate on that data together into a single, independent unit—an **object**. For the library example, we can create a `Book` class that contains **attributes** (data) like `title`, `author`, `isbn`, and **methods** (operations) like `checkOut()`, `returnBook()`. Then, each specific book (e.g., "The Three-Body Problem") is an **object** of this `Book` class.

The benefits are:

- * **Modularity**: Code is organized into independent, manageable modules (objects).
- * **Reusability**: The `Book` class can be used to create any number of book objects without rewriting code.
- * **Maintainability**: If you need to change a book's behavior, you only modify the `Book` class itself; all objects created from it are automatically updated.
- * **Extensibility**: You can easily add new classes (e.g., `Magazine`, `DVD`) to extend the program's functionality.

4.2 深入理解类与对象 / Deep Dive into Classes and Objects

中文

- **类 (Class) 是抽象的**: 它只是一个概念上的定义。例如, “汽车” 这个概念就是一个类。它定义了所有汽车都应该有的东西: 车轮、发动机、方向盘, 以及能做的事情: 启动、加速、刹车。但是, “汽车” 这个概念本身不能开, 它只是一个模板。

- **对象 (Object) 是具体的**：对象是类的一个实例，是真实存在的。例如，你停在楼下的那辆红色的特斯拉 Model 3 就是一个对象。它是根据“汽车”这个类创建出来的具体实例。它有具体的颜色（红色）、具体的品牌（特斯拉）、具体的型号（Model 3），并且你真的可以开它。

如何创建和使用对象？

在Java中，创建和使用对象通常遵循三个步骤：

1. **声明 (Declaration)**：声明一个变量，其类型是类名。这只是在栈内存中创建了一个引用，目前它还没有指向任何对象。

```
java // 声明一个名为 myCar 的变量，类型是 Car Car myCar;
```

2. **实例化 (Instantiation)**：使用 `new` 关键字来创建对象。`new` 关键字会在堆内存中为对象分配空间。

```
java // 使用 new 关键字创建一个 Car 对象 myCar = new Car();
```

3. **初始化 (Initialization)**：`new` 关键字后面跟着的 `Car()` 实际上是在调用 `Car` 类的**构造方法 (Constructor)**。构造方法负责初始化新创建的对象，比如给它的属性赋初始值。

```
java // 假设 Car 类有一个构造方法，可以设置颜色和型号 Car myCar = new Car("Red", "Tesla Model 3");
```

4. **使用 (Usage)**：一旦对象被创建，我们就可以通过 `.` 运算符来访问它的属性和方法。

```
java // 访问属性 System.out.println(myCar.color); // 输出: Red // 调用方法 myCar.start(); // 调用 Car 类的 start 方法 myCar.accelerate(); // 调用 Car 类的 accelerate 方法
```

English

- **Class is Abstract**: It's a conceptual definition. For example, the concept of "Car" is a class. It defines what all cars should have: wheels, an engine, a steering wheel, and what they can do: start, accelerate, brake. However, the concept of "Car" itself cannot be driven; it's just a template.
- **Object is Concrete**: An object is an instance of a class; it really exists. For example, the red Tesla Model 3 parked downstairs is an object. It's a

concrete instance created from the "Car" class. It has a specific color (red), a specific brand (Tesla), a specific model (Model 3), and you can actually drive it.

How to Create and Use Objects?

In Java, creating and using an object typically follows three steps:

1. **Declaration:** Declare a variable whose type is the class name. This only creates a reference in stack memory; it doesn't point to any object yet.

```
java // Declare a variable named myCar of type Car Car myCar;
```

2. **Instantiation:** Use the `new` keyword to create the object. The `new` keyword allocates memory for the object on the heap.

```
java // Use the new keyword to create a Car object myCar = new Car();
```

3. **Initialization:** The `Car()` following the `new` keyword is actually calling the **Constructor** of the `Car` class. The constructor is responsible for initializing the newly created object, e.g., giving its attributes initial values.

```
java // Assuming the Car class has a constructor that sets color and model
Car myCar = new Car("Red", "Tesla Model 3");
```

4. **Usage:** Once the object is created, we can access its attributes and methods using the `.` operator.

```
java // Access an attribute System.out.println(myCar.color); // Output: Red
// Call a method myCar.start(); // Calls the start method of the Car class
myCar.accelerate(); // Calls the accelerate method of the Car class
```

4.3 类的完整定义示例 / A Complete Class Definition Example

中文

让我们来看一个完整的 `Student` 类的例子，来巩固这些概念。

```
// 定义一个 Student 类
public class Student {
    // --- 属性 (Attributes / Fields) ---
    String name;
    int age;
```

```

String studentId;

// --- 构造方法 (Constructor) ---
// 这是一个无参构造方法
public Student() {
    // 可以在这里设置默认值
    this.name = "Unknown";
    this.age = 0;
    this.studentId = "N/A";
}

// 这是一个有参构造方法，用于初始化对象时设置属性
public Student(String name, int age, String studentId) {
    this.name = name;
    this.age = age;
    this.studentId = studentId;
}

// --- 方法 (Methods) ---
public void study() {
    System.out.println(name + " is studying.");
}

public void takeExam() {
    System.out.println(name + " is taking an exam.");
}

public void displayInfo() {
    System.out.println("Name: " + name + ", Age: " + age + ", ID: " +
studentId);
}
}

```

如何使用这个类？

```

```java
public class Main {
 public static void main(String[] args) {
 // 使用无参构造方法创建一个

```